Project Report

Student 1 Chi Hao Lu (19909400)
Student 2 Damian Nguyen (22049939)
Student 3 Qian Heng Yau (22028269)

Assignment 2 for COMP7023 Predictive Analytics
School of Computer, Data and Mathematical Sciences
Western Sydney University

Spring 2025

Declaration

By including this statement, we the authors of this work, verify that:

1. I hold a copy of this assignment that we can produce if the original is lost or damaged.
2. I hereby certify that no part of this assignment/product has been copied from any other student's work or from any other source except where due acknowledgement is made in the assignment.
3. No part of this assignment/product has been written/produced for us by another person except where such collaboration has been authorised by the subject lecturer/tutor concerned.
4. I am aware that this work may be reproduced and submitted to plagiarism detection software programs for the purpose of detecting possible plagiarism (which may retain a copy on its database for future plagiarism checking).
5. I hereby certify that we have read and understand what the School of Computer, Data and Mathematical Sciences defines as minor and substantial breaches of misconduct as outlined in the learning guide for this unit.

---

1 Introduction

Global warming has been the hot topic in the World Health Organisation (WHO) for many years. Vehicle emissions are a major source of environmental pollution and climate change, with carbon dioxide (CO2) and nitrogen oxides (NOX) being among the most significant pollutants. According to European Environment Agency (2023), CO2 has impacted the increase of temperature, while NOX emissions are associated with smog formation, acid rain, and adverse respiratory effects. Hence, the Intergovernmental Panel on Climate Change (IPCC) has suggested to closely monitor the gas emissions from the road vehicles.

This report analyses emissions using the data from the UK Vehicle Certification Agency (VCA), which provides verified measurements of vehicle fuel consumption and emissions under the Worldwide Harmonised Light Vehicle Test Procedure (WLTP). The VCA dataset ensures consistency and accuracy across vehicle testing in accordance with European and international standards (UK Vehicle Certification Agency 2024). Through statistical and predictive modelling, this study explores the key factors influencing emission levels to support sustainable transportation research and evidence-based environmental policymaking.

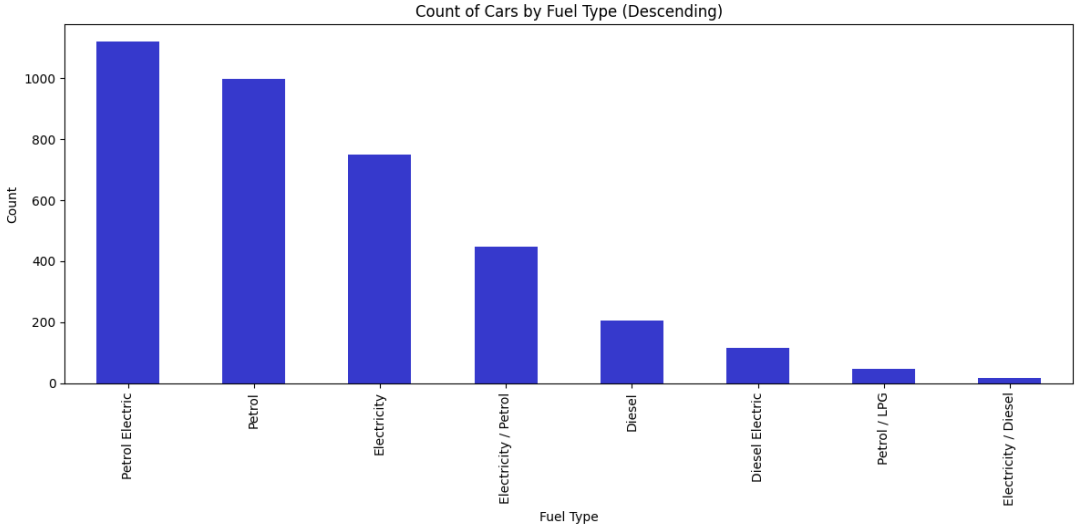
2 Exploratory Data Analysis

The data set contains of 3702 vehicles from different companies and 44 columns of measurement of vehicle types, fuel consumption and emmission.

Category	Columns
Vehicle	Manufacturer, Model, Description, Transmission, Manual or Automatic, Engine Capacity, Fuel Type, Powertrain, Engine Power (PS), Engine Power (Kw)
Fuel Consumption	Diesel VED Supplement, Testing Scheme, WLTP Imperial Low, WLTP Imperial Medium, WLTP Imperial High, WLTP Imperial Extra High, WLTP Imperial Combined, WLTP Imperial Combined (Weighted), WLTP Metric Low, WLTP Metric Medium, WLTP Metric High, WLTP Metric Extra High, WLTP Metric Combined, WLTP Metric Combined (Weighted)
Electric Consumption	Electric energy consumption Miles/kWh, wh/km ,Maximum range (Km), Maximum range (Miles), Equivalent All Electric Range Miles, Equivalent All Electric Range KM, Electric Range City Miles, Electric Range City Km
Standard	Euro Standard
Emmission	WLTP CO2, WLTP CO2 Weighted, Emissions CO [mg/km], THC Emissions [mg/km], Emissions NOx [mg/km], THC + NOx Emissions [mg/km], Particulates [No.] [mg/km], RDE NOx Urban, RDE NOx Combined, Noise Level dB(A)
Date	Date of change

Category	Columns

Out of the 44 columns only 9 columns which are Manufacturer, Model, Description, Transmission, Manual or Automatic, Fuel Type, Powertrain, Euro Standard, and Testing Scheme are the non-numeric data type, the rest of columns are numeric or boolean which can be used modelling. From following chart, we can see the distribution of fuel type of the vehicle in the data set. We can group the vehicles into three main categories: Fuel, Hybrid and Electric.

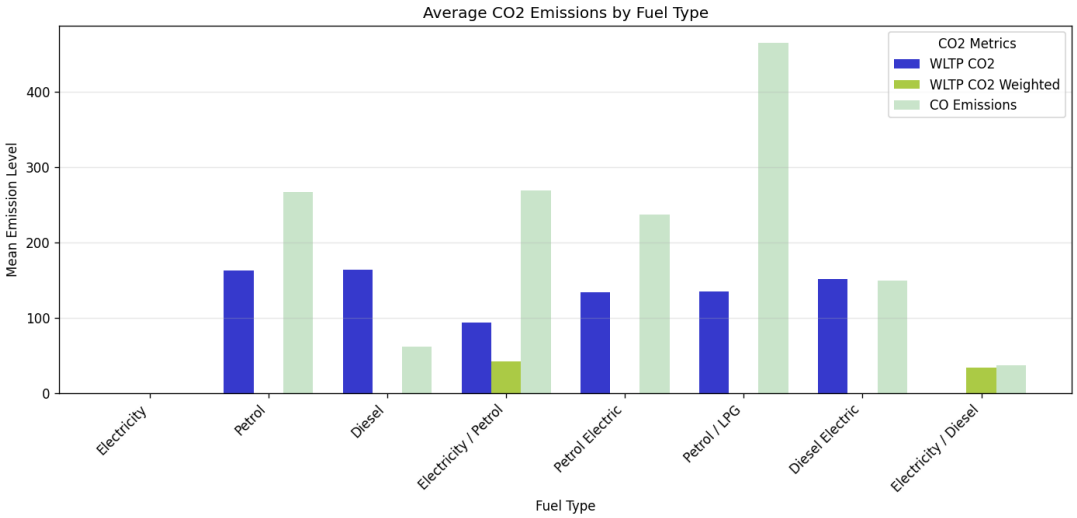


2.1 Emissions Analysis

As the discussion is about emission, we will further understand the different types of emissions such as Carbon Monoxide (CO), Carbon Dioxide (CO2), Nitrogen Oxides (NOx) and Total Hydrocarbons (THC). These emissions are not polluting the environment but impacting our health. According to WHO (2021), when CO binds with haemoglobin more readily than oxyegen, it might reduce oxygen transport in blood and posing serious health risks to humans. While CO2 is non-toxis to humans but it significantly contributes to global warming and climate change as it can trap the heat in the atmosphere (Intergovernmental Panel on Climate Change [IPCC], 2021). When the temperature increase, it might bring disasters to human such as flood or extreme heat wave.

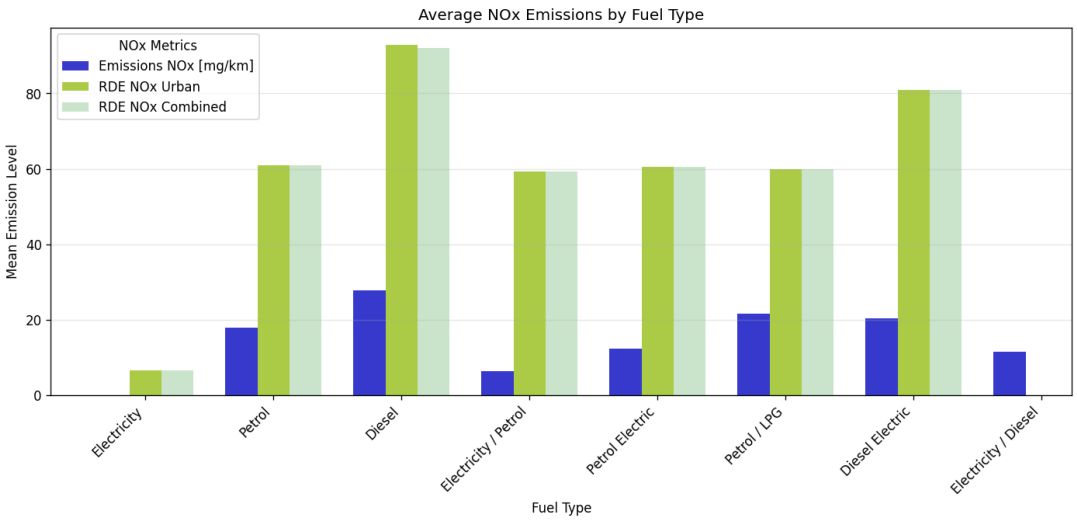
2.1.1 Carbon Monoxide (CO) and Carbon Dioxide (CO2)

From the following bar chart, we can clearly see the emissions of CO2 and CO are co-exists in petrol or hybrid vehicles but not in electric car. Emission of CO is high especially in diesel vehicle and emission of CO2 based on WLTP CO2 is about the same in petrol or diesel verhicles but low in hybrid vehicles. WLTP CO2 Weighted seems to be not reliable as it is empty for most of the vehicles type.



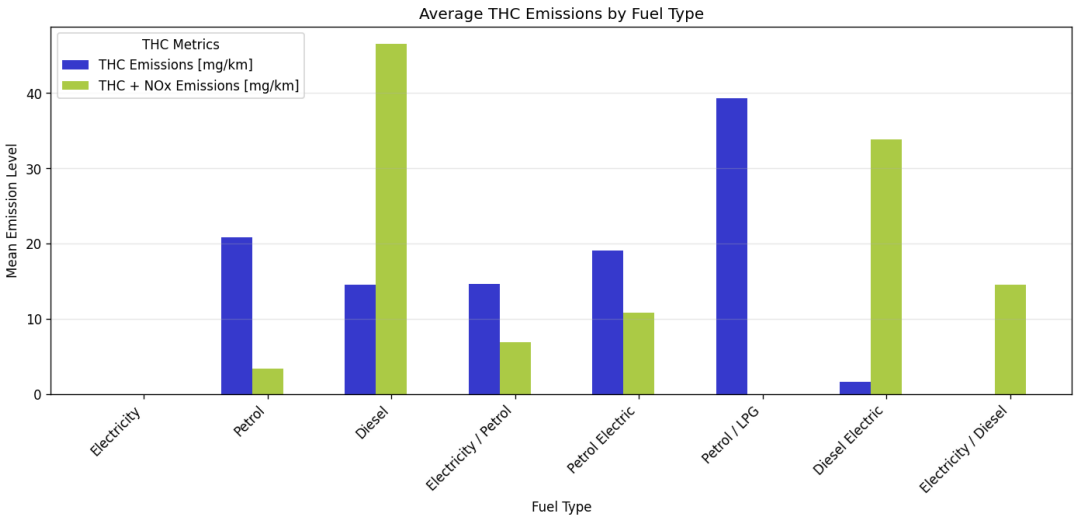
2.1.2 Nitrogen Oxides (NOx)

Nitrogen Oxides (NOx) emissions are a major cause to photochemical smog, acid rain, and respiratory health issues (U.S. Environmental Protection Agency [EPA], 2023). Based on following chart, we find NOx emission exists in all the vehicles including electic car. Electric cars do produce a small average amount of NOx according Veza et al (2023). However, it should be understood that the NOx production from electric vehicles is caused from the source of electric used to charge the car batteries. NO emissions from electric vehicles originate from the fossil fuel–based electricity used to charge them. The mean for NOx in Real Driving Emissions (RDE) for urban and combined are about same but they are higher than Emissions (NOx) [mg/km] which was the result tested in lab.



2.1.3 Total Hydrocarbons (THC)

Total Hydrocarbons (THC) are volatile organic compounds (VOCs) that react with NOx in the presence of sunlight to form ground-level ozone and smog (European Environment Agency [EEA], 2022). THC might cause photochemical pollution and impact the urban air quality but it have to react with NOx, hence, in this report we will not focus in analysing the relationship of THC with other features.



2.2 Target Variables

From the analysis, we have enough data for modelling accross different fuel types but we will choose CO2 as global warming is most common topic when discussing about the impacts of emission. As mentioned above, WLTP CO2 Weighted is lacking of data distribution accross fuel types, hence it will be excluded from target variables. Besides of CO2, we will pick Emissions NOx [mg/km] as our target variables because it has difference mean across fuld type which is easier for our comparison and prediction afterward in this report.

Based on the graphs above, eletric cars have no CO2 emissions, and low NOx emissions. Therefore, all of the eletric cars will be excluded from the datasets when we build the model. Given the distribution, two datasets will be created based on their fuel types - fuel cars only (data_fuel), and hybrid cars only (data_hybrid). Furthermore, since the target variables (WLTP CO2 and NOx emissions) are not highly included in Eletricity /Diesel Fuel type, they will be removed from the data_hybrid.

3 Data Preprocessing

3.1 Model Building

After identified the target variables - Emissions NOx [mg/km] and WLTP CO2, based on the category that we have defined above, we will choose what are the relevant columns to keep in the modelling and remove all irrelevant columns. Reducing the variables/columns can help in improving the performance and cost effectiveness. Variables that are related to the manufacturer or car model will be removed as these variables cannot be used in predictions. As mentioned above, we will create 2 datasets - data_fuel and data_hybrid, which contains fuel vehicles and

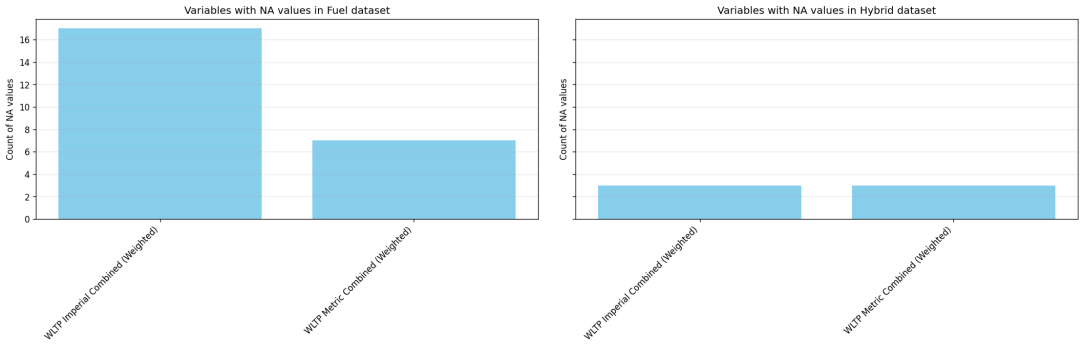
hybrid vehicles. Hence, the variables that are not relevant to fuel or hybrid will be removed too. For emmissions, we will keep only Emissions NOx [mg/km] and WLTP CO2 as our target variables and drop rest of the emissions related columns. Lastly, date and standard category columns will be removed too as this information cannot be used for modeling.

Category	Keep	Remove
Vehicle	Manual or Automatic, Engine Capacity, Fuel Type, Powertrain, Engine Power (PS)	Manufacturer, Model, Description, Transmission, Engine Power (Kw)
Fuel Consumption	WLTP Imperial Low, WLTP Imperial Medium, WLTP Imperial High, WLTP Imperial Extra High, WLTP Imperial Combined, WLTP Imperial Combined (Weighted), WLTP Metric Low, WLTP Metric Medium, WLTP Metric High, WLTP Metric Extra High, WLTP Metric Combined, WLTP Metric Combined (Weighted)	Diesel VED Supplement, Testing Scheme
Electric Consumption		Electric energy consumption Miles/kWh, wh/km ,Maximum range (Km), Maximum range (Miles), Equivalent All Electric Range Miles, Equivalent All Electric Range KM, Electric Range City Miles, Electric Range City Km
Standard		Euro Standard
Emmission	Emissions NOx [mg/km], WLTP CO2	WLTP CO2 Weighted , THC Emissions [mg/km], THC + NOx Emissions [mg/km], Particulates [No.] [mg/km], RDE NOx Urban, RDE NOx Combined, Noise Level dB(A)
Date		Date of change

After removing the irrelevant variables, we have 19 variables remained in the datasets. Next, we will split our data into two groups as mentioned above which are the hybrid vehicles and fuel vehicles. We are getting **1686 hybrid** vehicles and **1250 fuel** vehicles in our datasets after splitting by the fuel type.

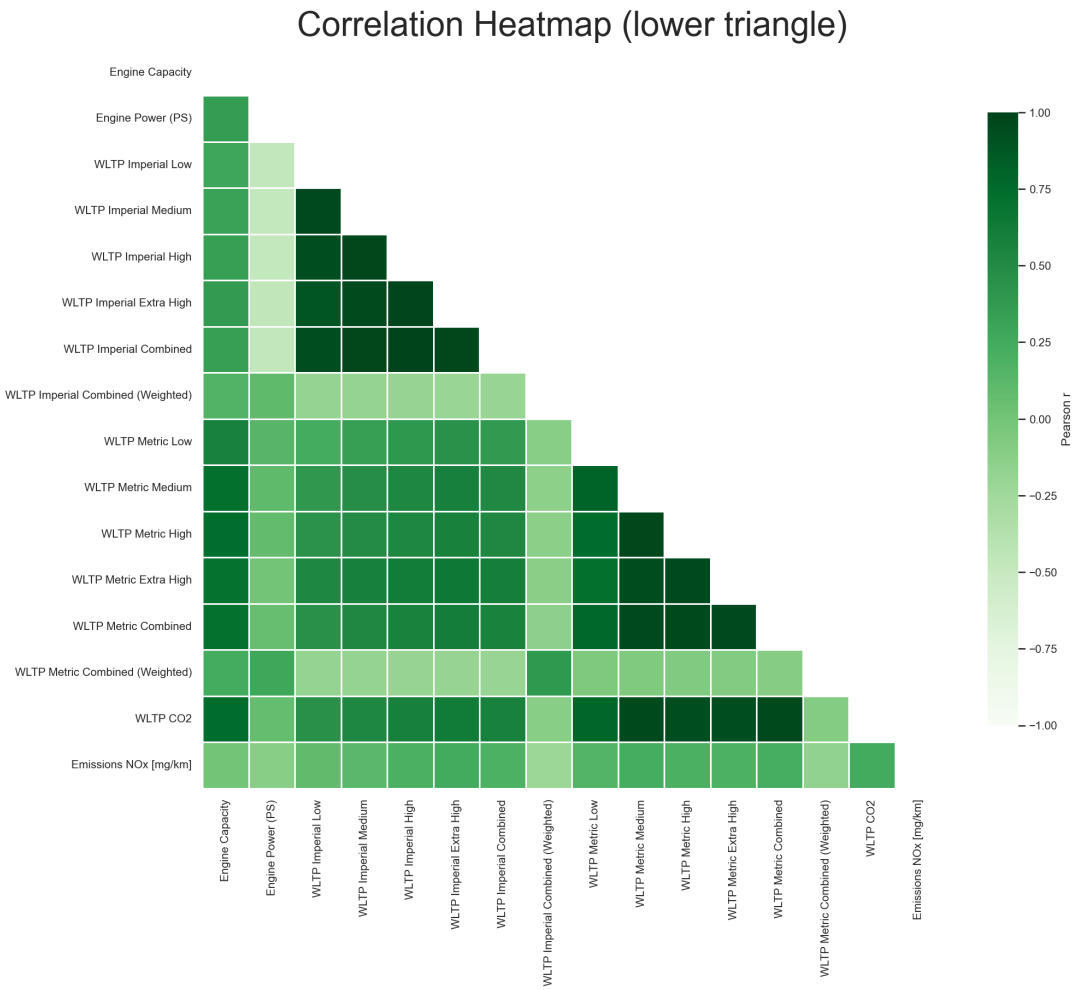
3.2 Data Cleaning

Before we start the modelling, cross checking on the data with "not available (NA)" values or empty, this will help to ensure the accuracy of our model. Based on the result, we found NA values in WLTP Imperial Combined (Weighted) and WLTP Metric Combined (Weighted) columns in both fuel and hybrid datasets. After removing the NA rows, we have **1233 fuel** vehicles and **1683 hybrid** vehicles remained in the datasets.

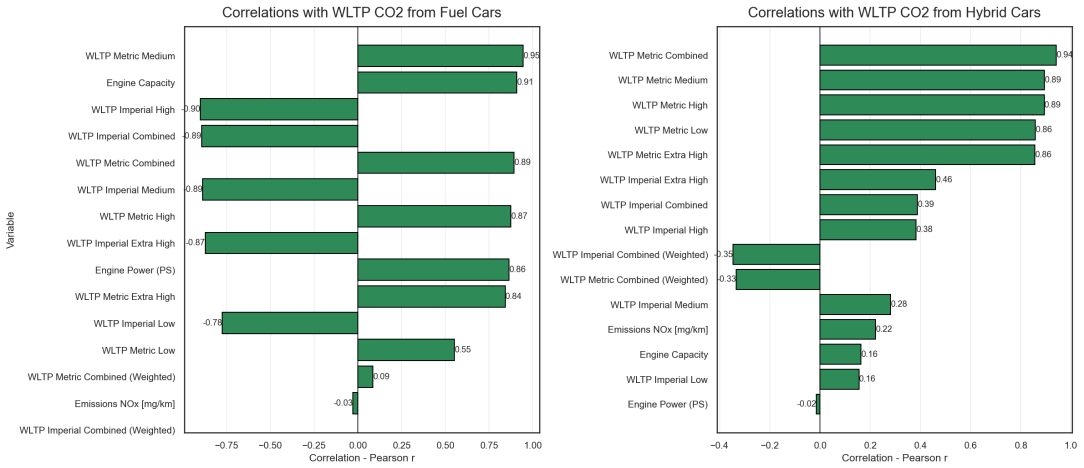


3.3 Correlation Analysis

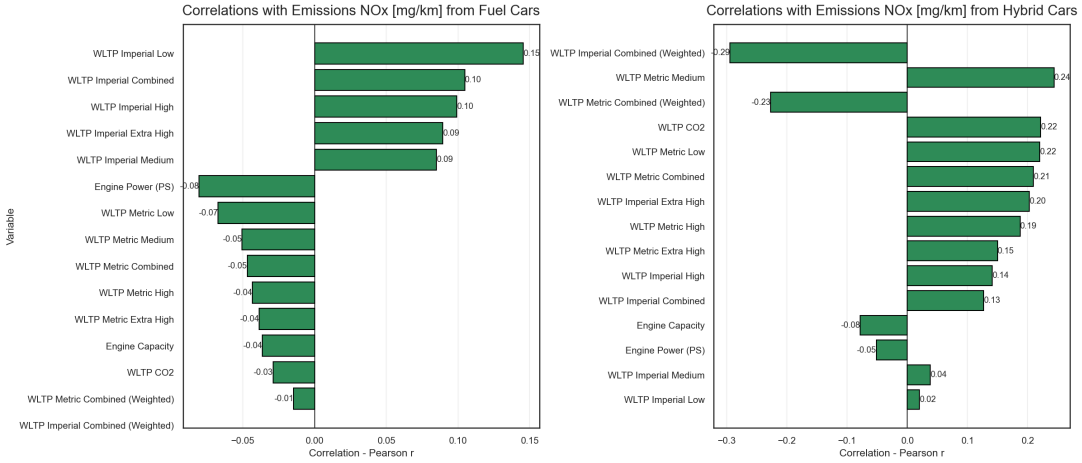
Correlation analysis started with a heatmap to understand the correlation between each variables. From the heatmap, we can see the correlation with our target variables are low at Engine Power (Ps), WLTP Imperial Combined (Weighted), and WLTP Metric Combined (Weighted).



When drilling down to the correlations with CO2, we can see that the WLTP Imperial Combined (Weighted) and WLTP Metric Combined (Weighted) are having the lowest score in both fuel and hybrid vehicles. Whereas fuel is having high correlation with fuel vehicle but not the hybrid vehicle.

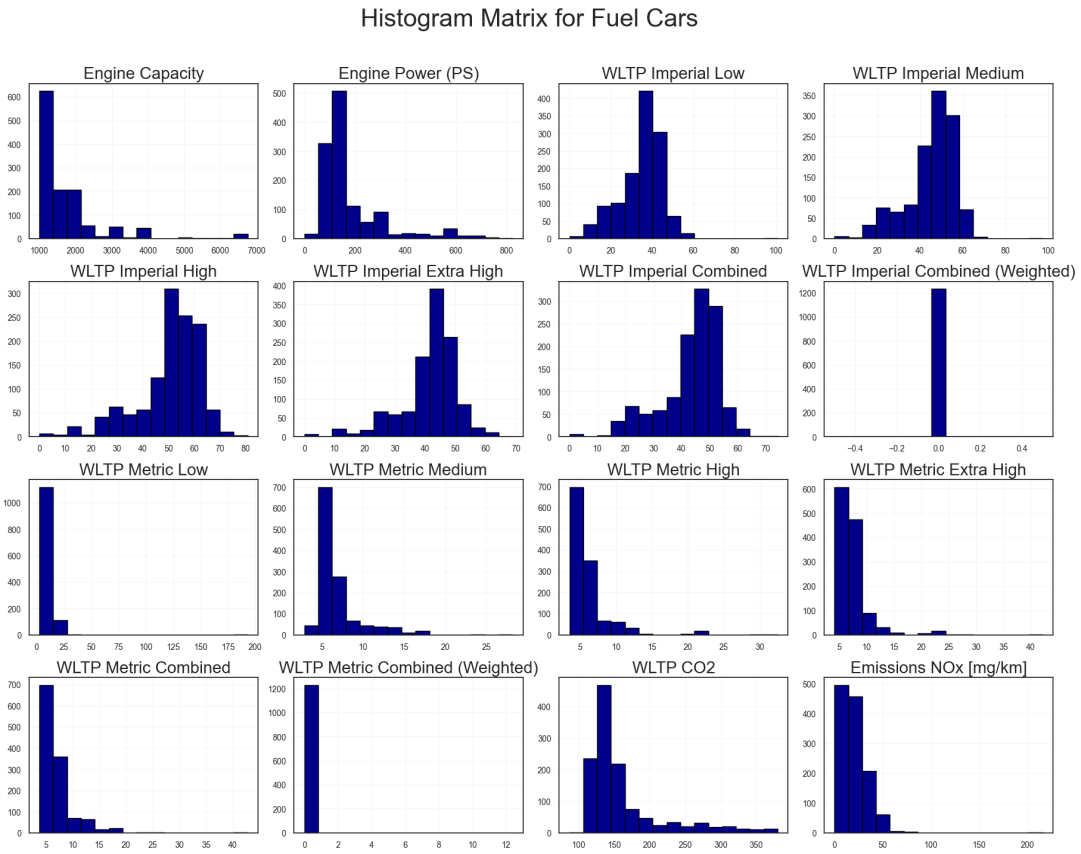


In NOx emmissions, WLTP Imperial Combined (Weighted) and WLTP Metric Combined (Weighted) are still having lowest correlation in fuel vehicles but these variables are the top three correlated variables in hybrid vehicles. This could be due to the hybrid engine turns on intermittently, it often operates under non-steady, high-load conditions to recharge the battery or accelerate.

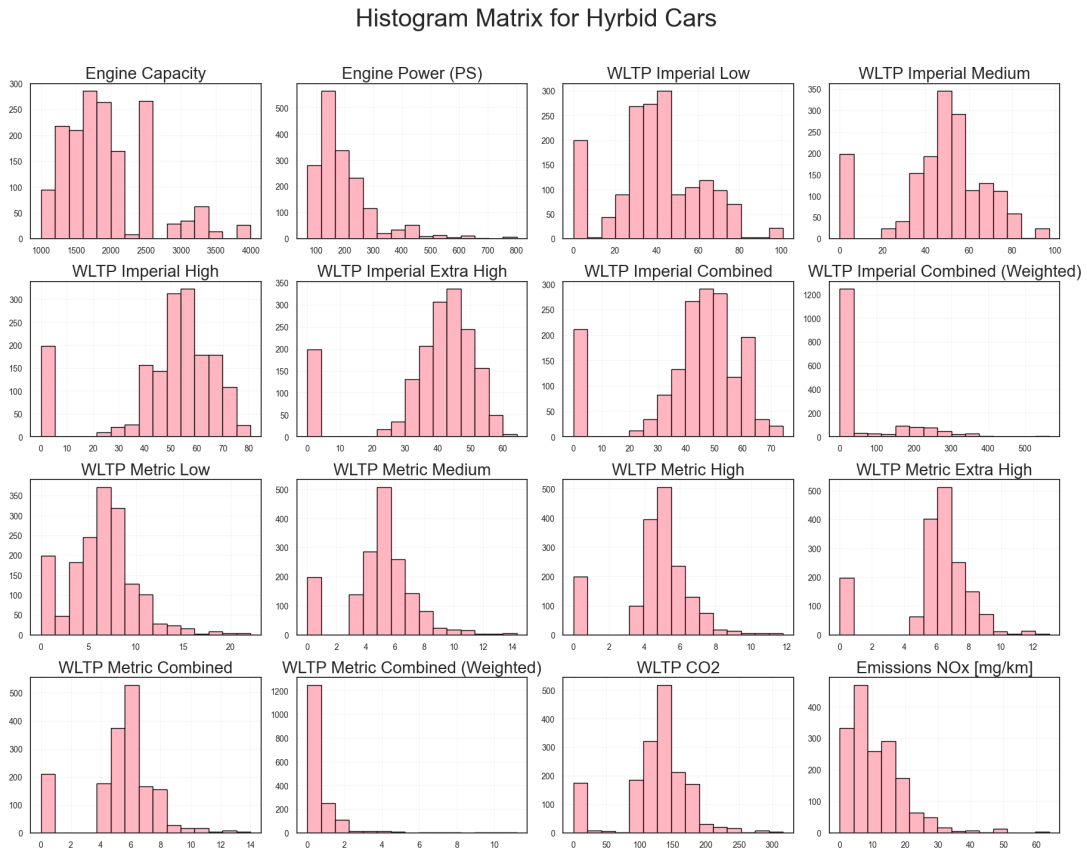


3.4 Data Distribution Analysis

Distribution of data is important in identifying the normalisation or standardisation of each variables in our datasets. Below histogram matrix that showing the data distribution for fuel vehicles, we can see that WLTP Imperial Combined (Weighted) and WLTP Metric Combined (Weighted) are not statistically normalised, as their disbutions are tightly clustered and do not follow a standardisation or z-score normalisation. Hence, we can safely remove WLTP Imperial Combined (Weighted) and WLTP Metric Combined (Weighted) columns from *data_fuel* dataset



Each variable in hybrid vehicles is statistically normalised, we do not see abnormal data distribution here. Hence, no changes to the variables for hybrid vehicles.

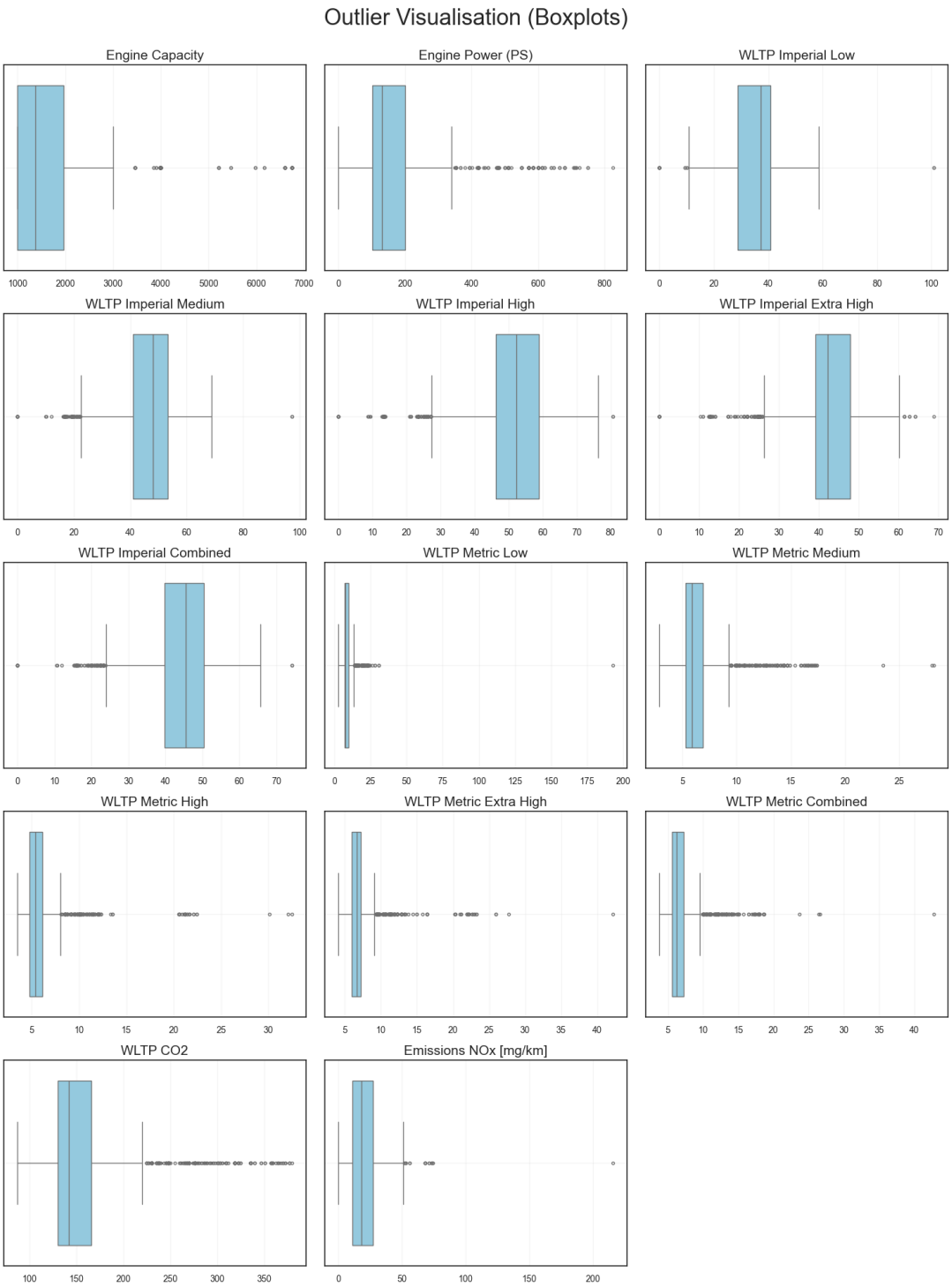


3.5 Outlier Check

To further improve our model and increase the accuracy, we have perform an analysis of the outliers for both datasets. In the fuel vehicles, we can clearly see the outliers are strongly skewed in most of the WLTP Metric variables and highly skewed in Engine Capacity and Power. This could be due to the high-performance models of vehicles. Since, the WLTP Imperial is having the same purpose with a different unit of measurements, we have decided to drop all the WLTP Metric variables which having higher outliers compare to MLTP Imperial. This table showing the updated model for *data_fuel*.

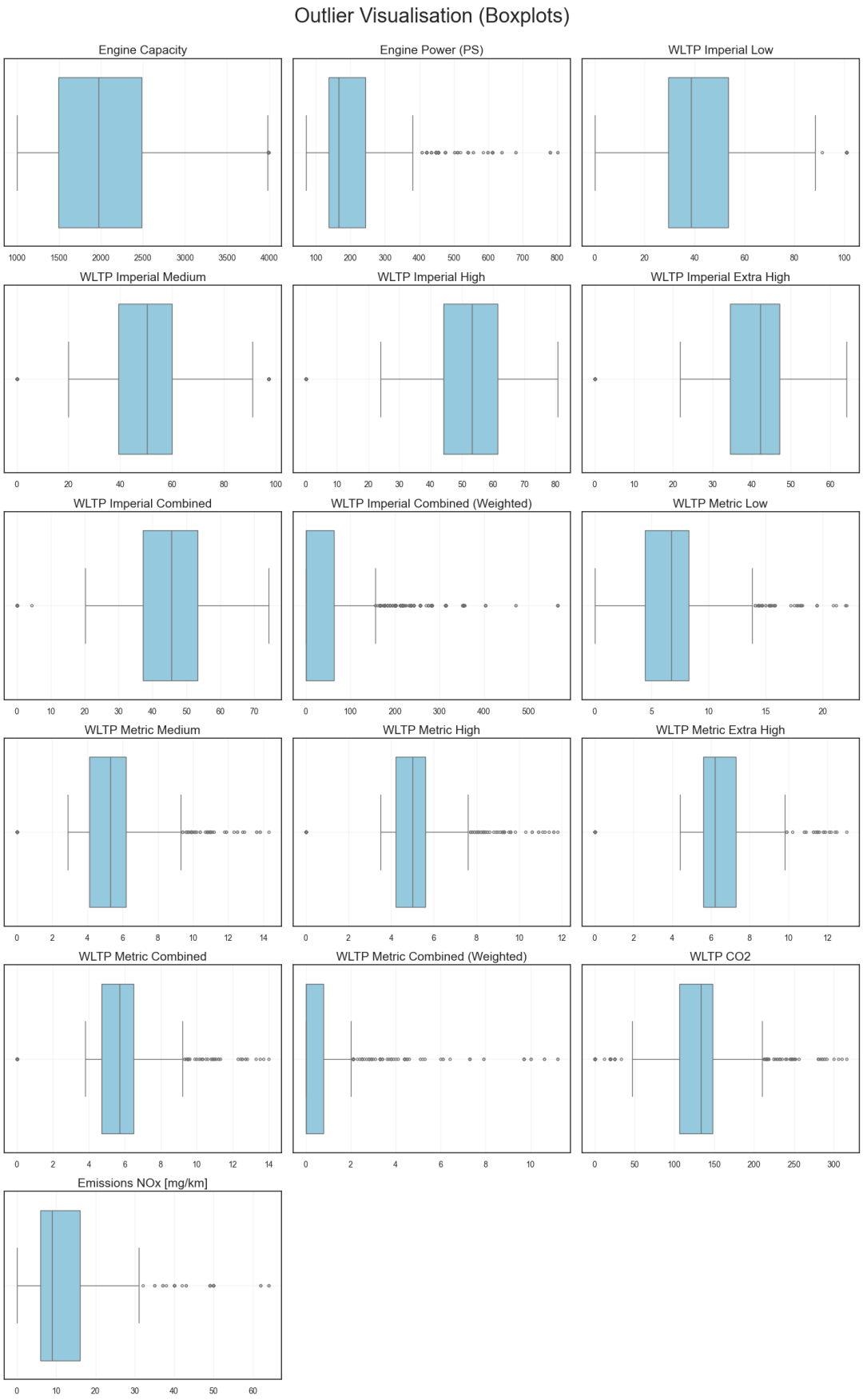
Category	Keep	Remove
Vehicle	Manual or Automatic, Engine Capacity, Fuel Type, Powertrain, Engine Power (PS)	Manufacturer, Model, Description, Transmission, Engine Power (Kw)

Category	Keep	Remove
Fuel Consumption	WLTP Imperial Low, WLTP Imperial Medium, WLTP Imperial High, WLTP Imperial Extra High, WLTP Imperial Combined	Diesel VED Supplement, Testing Scheme, WLTP Imperial Combined (Weighted), WLTP Metric Combined (Weighted), WLTP Metric Low, WLTP Metric Medium, WLTP Metric High, WLTP Metric Extra High, WLTP Metric Combined
Electric Consumption		Electric energy consumption Miles/kWh, wh/km ,Maximum range (Km), Maximum range (Miles), Equivalent All Electric Range Miles, Equivalent All Electric Range KM, Electric Range City Miles, Electric Range City Km
Standard Emmission	Emissions NOx [mg/km], WLTP CO2	Euro Standard WLTP CO2 Weighted , THC Emissions [mg/km], THC + NOx Emissions [mg/km], Particulates [No.] [mg/km], RDE NOx Urban, RDE NOx Combined, Noise Level dB(A)
Date		Date of change



Same for fuel vehicles, it seems to be a high outliers in WLTP Metric for hybrid vehicles. Therefore, these variables will be removed from hybrid vehicles dataset too. Here is the table showing the updated columns remain in *data_hybrid*.

Category	Keep	Remove
Vehicle	Manual or Automatic, Engine Capacity, Fuel Type, Powertrain, Engine Power (PS)	Manufacturer, Model, Description, Transmission, Engine Power (Kw)
Fuel Consumption	WLTP Imperial Low, WLTP Imperial Medium, WLTP Imperial High, WLTP Imperial Extra High, WLTP Imperial Combined	Diesel VED Supplement, Testing Scheme, WLTP Metric Low, WLTP Metric Medium, WLTP Metric High, WLTP Metric Extra High, WLTP Metric Combined, WLTP Metric Combined (Weighted), WLTP Imperial Combined (Weighted)
Electric Consumption		Electric energy consumption Miles/kWh, wh/km ,Maximum range (Km), Maximum range (Miles), Equivalent All Electric Range Miles, Equivalent All Electric Range KM, Electric Range City Miles, Electric Range City Km
Standard		Euro Standard
Emmission	Emissions NOx [mg/km], WLTP CO2	WLTP CO2 Weighted , THC Emissions [mg/km], THC + NOx Emissions [mg/km], Particulates [No.] [mg/km], RDE NOx Urban, RDE NOx Combined, Noise Level dB(A)
Date		Date of change


Outlier Visualisation (Boxplots)
Engine Capacity
1000 1500 2000 2500 3000 3500 4000
Engine Power (PS)
100 200 300 400 500 600 700 800
WLTP Imperial Low
0 20 40 60 80 100
WLTP Imperial Medium
0 20 40 60 80 100
WLTP Imperial High
0 10 20 30 40 50 60 70 80
WLTP Imperial Extra High
0 10 20 30 40 50 60
WLTP Imperial Combined
0 10 20 30 40 50 60 70
WLTP Imperial Combined (Weighted)
0 100 200 300 400 500
WLTP Metric Low
0 5 10 15 20
WLTP Metric Medium
0 2 4 6 8 10 12 14
WLTP Metric High
0 2 4 6 8 10 12
WLTP Metric Extra High
0 2 4 6 8 10 12
WLTP Metric Combined
0 2 4 6 8 10 12 14
WLTP Metric Combined (Weighted)
0 2 4 6 8 10
WLTP CO2
0 50 100 150 200 250 300
Emissions NOx [mg/km]
0 10 20 30 40 50 60

3.6 Cross Validation

Now, we have the final model with all the features and target variables to train and test and model. Before we start the model, a *data_split* function was created to split the *data_fuel* and *data_hybrid* into train, validate, and test datasets. The datasets were split into **80% for training, 10% for validations and 10% for testing**.

After splitting the data, we have created a function called *train_lasso_with_cv* to use **Lasso** model for feature selection with a **10-fold** cross validations. Based on the results from Lasso model, we have designed to refine the Lasso model again with the best L1 regularisation strength (*alphas*). With the results from Lasso model, we can understand the errors using **mean squared error (MSE)** and variances by calculating the **r2_score** as well as listing the coefficient of each features.

We have done the training for both datasets *data_fuel* and _data_hybrid with the two target variables *WLTP CO2* and *Emissions NOx [mg/km]* that we have defined above. Below table is the results from the *lasso* model training with *10-fold* cross validation.

Metric	Fuel / CO2 (12 features)	Fuel / CO2 (3 features)	Fuel / NOx (12 features)	Fuel / NOx (3 features)	Hybrid / CO2 (12 features)	Hybrid / CO2 (3 features)	Hybrid / NOx (12 features)	Hybrid / NOx (3 features)
Best alpha	0.054	0.022	0.022	0.002	0.103	0.022	0.004	0.055
Train RMSE	197.082	2071.979	213.583	238.598	1359.811	2071.979	45.390	74.931
Validate RMSE	270.483	1912.145	1272.888	171.254	1044.115	1912.145	71.701	98.815
Validate R^2	0.926	0.055	-6.459	-0.003	0.484	0.055	0.279	0.007
Test RMSE	247.201	2080.948	1172.173	149.236	1525.036	2080.948	24.205	45.544
Test R^2	0.890	0.251	6.831	0.002	0.451	0.251	0.463	-0.009

Based on the *alphas* returned from first round of training, we have run the *lasso* model again with following results.

Metric (refined)	Fuel / CO2 (12 features)	Fuel / CO2 (3 features)	Fuel / NOx (12 features)	Fuel / NOx (3 features)	Hybrid / CO2 (12 features)	Hybrid / CO2 (3 features)	Hybrid / NOx (12 features)	Hybrid / NOx (3 features)
Validate RMSE	261.047	1912.280	1276.104	171.228	1043.926	1912.280	71.691	98.813
Validate R^2	0.928	0.055	-6.478	-0.003	0.484	0.055	0.279	0.007
Test RMSE	236.887	2081.085	1175.241	149.221	1525.295	2081.085	24.225	45.544
Test R^2	0.894	0.251	-6.852	0.003	0.451	0.251	0.462	-0.009

From the *lasso* training, we found that the model with all the 12 features are working fine in both datasets for fuel and hybrid vehicle when we examine the correlation with CO2 and NOx. The errors have increased drastically after reduced to 3 features only. Eventhough the model for Fuel/NOx was strongly negative in validation R2, we are keeping the 12 features to standardised the features selection. We will consider an algorithm improvements in the future.

3.7 Neural Network - Regression Tasks

Variables Selection The following variables should be used to develop neural networks model.

[38]:
```
#print current variables from the fuel cars dataset
print(data_fuel.columns)
print(data_fuel.shape)
```

```
Index(['Manual or Automatic', 'Engine Capacity', 'Fuel Type', 'Powertrain',
       'Engine Power (PS)', 'WLTP Imperial Low', 'WLTP Imperial Medium',
       'WLTP Imperial High', 'WLTP Imperial Extra High',
       'WLTP Imperial Combined', 'WLTP CO2', 'Emissions NOx [mg/km]'],
      dtype='object')
(1233, 12)
```

3.7.1 Neural Network Regression (2 Layers)

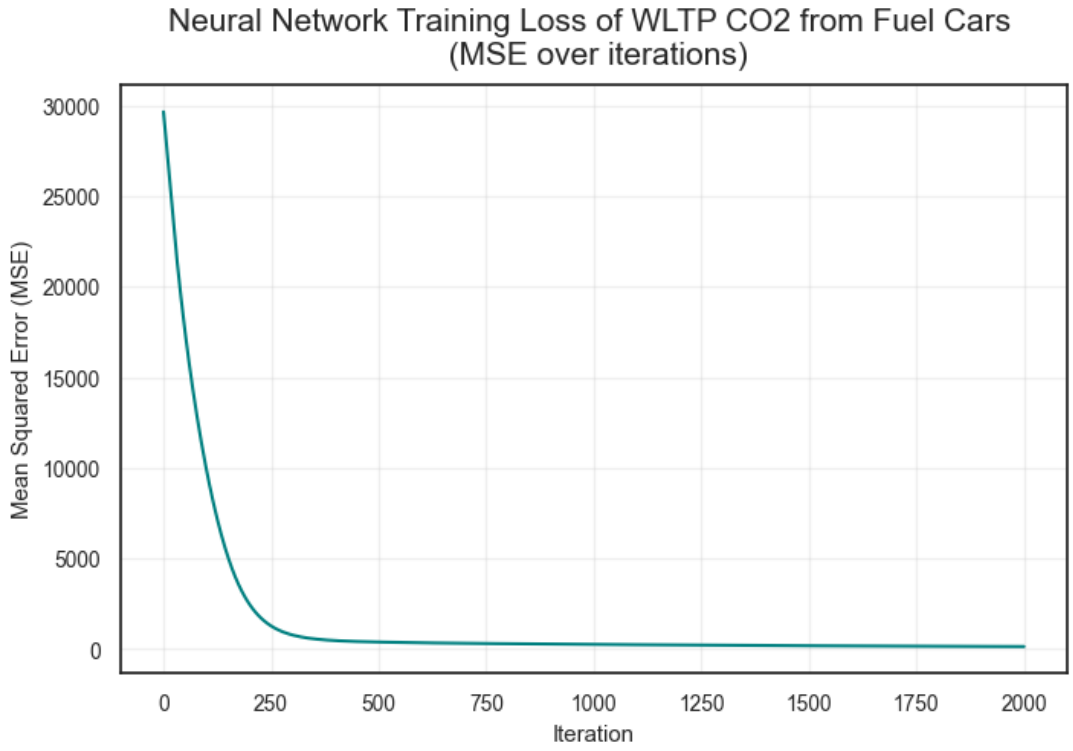
[39]:
```
# Reusable function for Neural Network regression task - simple 1 hidden␣
 ↪layer NN
def neural_network_regression(target: str, data: pd.DataFrame, random_state:␣
 ↪int = 42):
    assert target in data.columns, f"Target '{target}' not in dataframe."

    #reuse this part to access the splitted data.
    splits = data_split(target, data)

    # Access the data
    X_train = splits["X_train"]
    X_val   = splits["X_val"]
    X_test  = splits["X_test"]
    y_train = splits["y_train"]
    y_val   = splits["y_val"]
    y_test  = splits["y_test"]

    # Flatten target arrays
    y_train = y_train.values.reshape(-1, 1)
    y_val   = y_val.values.reshape(-1, 1)
    y_test  = y_test.values.reshape(-1, 1)

    # Define NN dimensions
    N = len(y_train) # number of samples
    D_in = X_train.shape[1] # number of features - 12
    D_out = 1 # number of outputs
    H = 20 # number of hidden neurons, based on number of features
```

```
    # Initialize weights
    W0 = normal(0, 0.1, size=(D_in, H))
    W1 = normal(0, 0.1, size=(H, D_out))

    # Training loop
    eta = 0.0005

    loss_history = []
    for i in range(2001):
        # forward pass

        ## Bias Terms
        b0 = np.zeros((1, H))
        b1 = np.zeros((1, 1))
        # z1 = X.dot(W0) + b0
        # a2 = a1.dot(W1) + b1

        z1 = X_train.dot(W0) + b0
        z1 = np.clip(z1, -500, 500)
        a1 = 1 / (1 + np.exp(-z1))
        a2 = a1.dot(W1) + b1
        Loss = np.mean((a2 - y_train) ** 2)
        loss_history.append(Loss)

        # # print loss every 100 iterations
        # if i % 100 == 0:
        #     print(f"Iteration {i}: Loss {Loss:.4f}")

        # backward pass
        da2 = 2.0 * (a2 - y_train)
        dW1 = a1.T.dot(da2) / N
        da1 = da2.dot(W1.T)
        dz1 = da1 * a1 * (1 - a1)
        dW0 = X_train.T.dot(dz1) / N

        # update
        W0 -= eta * dW0 # weights update for features
        W1 -= eta * dW1

    # Evaluate
    z1 = X_train.dot(W0)
    z1 = np.clip(z1, -500, 500)
    a1 = 1 / (1 + np.exp(-z1))
    y_pred = a1.dot(W1)

    # MSE and RMSE on training data
    print('Training RMSE:', np.sqrt(mean_squared_error(y_train, y_pred)))
    print('Coefficient of determination (R²) on training data:',␣
↪round(r2_score(y_train, y_pred),4))

    # Evaluate on test set
```

```
    z1 = X_test.dot(W0)
    z1 = np.clip(z1, -500, 500)
    a1 = 1 / (1 + np.exp(-z1))
    y_test_pred = a1.dot(W1)

    print('Test RMSE:', np.sqrt(mean_squared_error(y_test, y_test_pred)))
    print('Coefficient of determination (R²) on test data:',␣
↪round(r2_score(y_test, y_test_pred),4))

    # Evaluate on validation set
    z1 = X_val.dot(W0)
    z1 = np.clip(z1, -500, 500)
    a1 = 1 / (1 + np.exp(-z1))
    y_val_pred = a1.dot(W1)
    print('Validation RMSE:', np.sqrt(mean_squared_error(y_val, y_val_pred)))
    print('Coefficient of determination (R²) on  validation data:',␣
↪round(r2_score(y_val, y_val_pred),4))


    # Auto labeling based on Fuel Type column
    if data["Fuel Type"].isin(fuel_types).any():
        data_label = "Fuel Cars"
    elif data["Fuel Type"].isin(hybrid_types).any():
        data_label = "Hybrid Cars"
    else:
        data_label = "Dataset"


    #Visualise the MSE
    plt.figure(figsize=(8,5))
    plt.plot(loss_history, color='teal')
    plt.title(f"Neural Network Training Loss of {target} from {data_label}␣
↪\n (MSE over iterations)",
              fontsize = 15, pad = 10)
    plt.xlabel("Iteration")
    plt.ylabel("Mean Squared Error (MSE)")
    plt.grid(alpha=0.3)
    plt.show()

    # # Return the weights and features of the model
    # return{
    #     "features": features,
    #     "W0": W0
    #     }

# Neural Network regression for fuel cars, target variable: WLTP CO2
neural_network_regression("WLTP CO2", data_fuel)
```

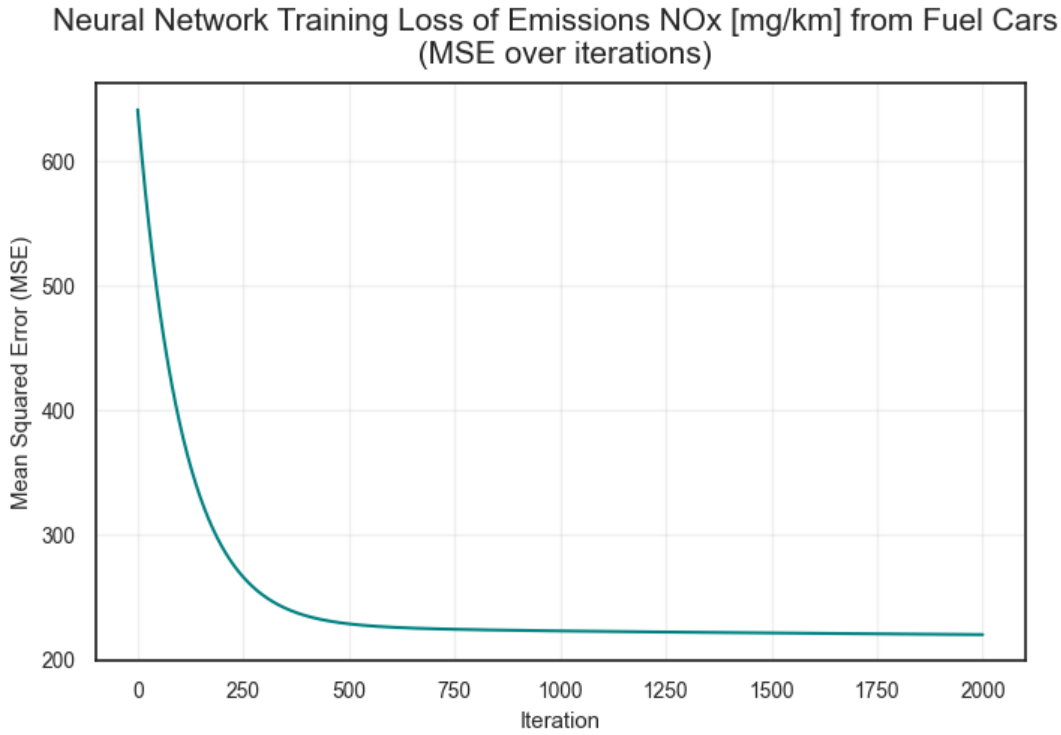
```
Training RMSE: 11.83214538566079
Coefficient of determination (R²) on training data: 0.9541
Test RMSE: 43.92853084963061
```

```
Coefficient of determination (R²) on test data: 0.1397
Validation RMSE: 57.19261359806857
Coefficient of determination (R²) on  validation data: 0.1014
```



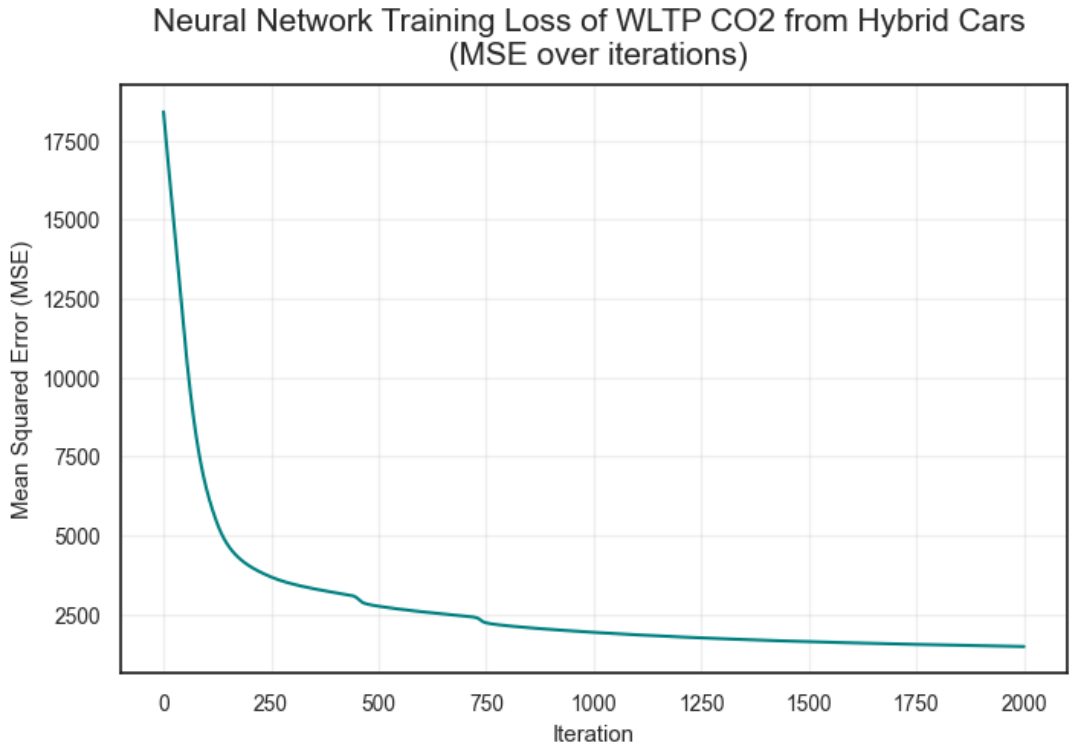
[40]:
```
# Neural Network regression for fuel cars, target variable: Emissions NOx
↪[mg/km]
neural_network_regression("Emissions NOx [mg/km]", data_fuel)
```

```
Training RMSE: 14.821462889091457
Coefficient of determination (R²) on training data: 0.1043
Test RMSE: 11.84380327307547
Coefficient of determination (R²) on test data: 0.0628
Validation RMSE: 12.432183269388801
Coefficient of determination (R²) on  validation data: 0.0942
```



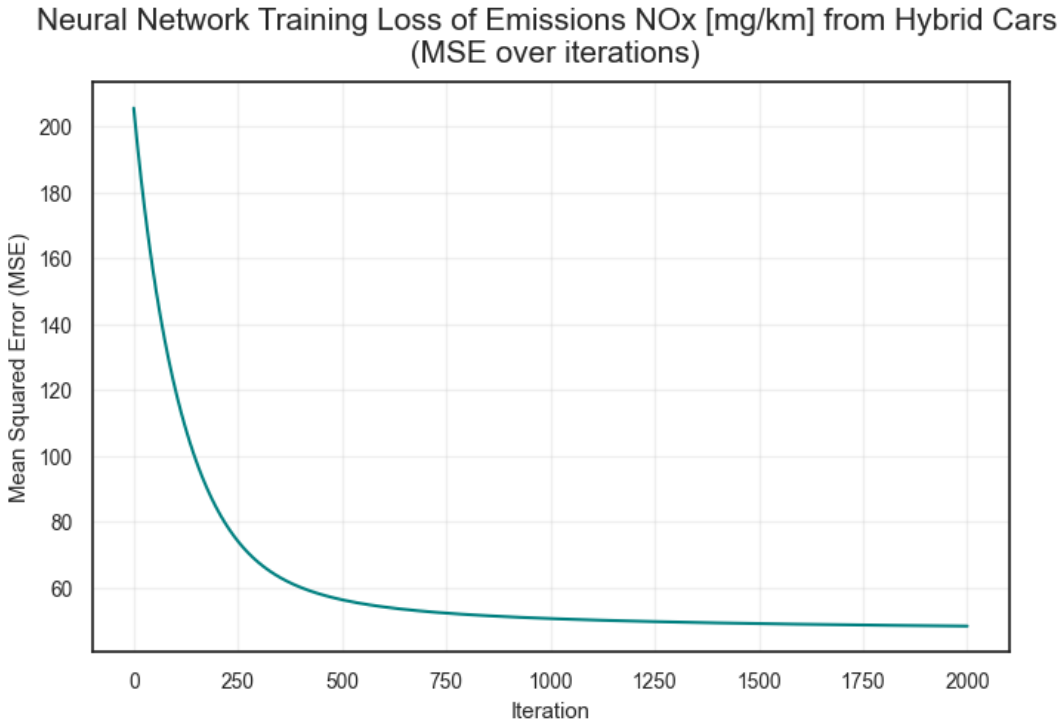
[41]:
```
# Neural Network regression for hybrid cars, target variable: WLTP CO2
neural_network_regression("WLTP CO2", data_hybrid)
```

```
Training RMSE: 38.45094043133957
Coefficient of determination (R²) on training data: 0.4958
Test RMSE: 38.71394959730473
Coefficient of determination (R²) on test data: 0.4605
Validation RMSE: 30.797058493922243
Coefficient of determination (R²) on  validation data: 0.5314
```



[42]:
```
# Neural Network regression for hybrid cars, target variable: Emissions NOx [mg/km]
neural_network_regression("Emissions NOx [mg/km]", data_hybrid)
```

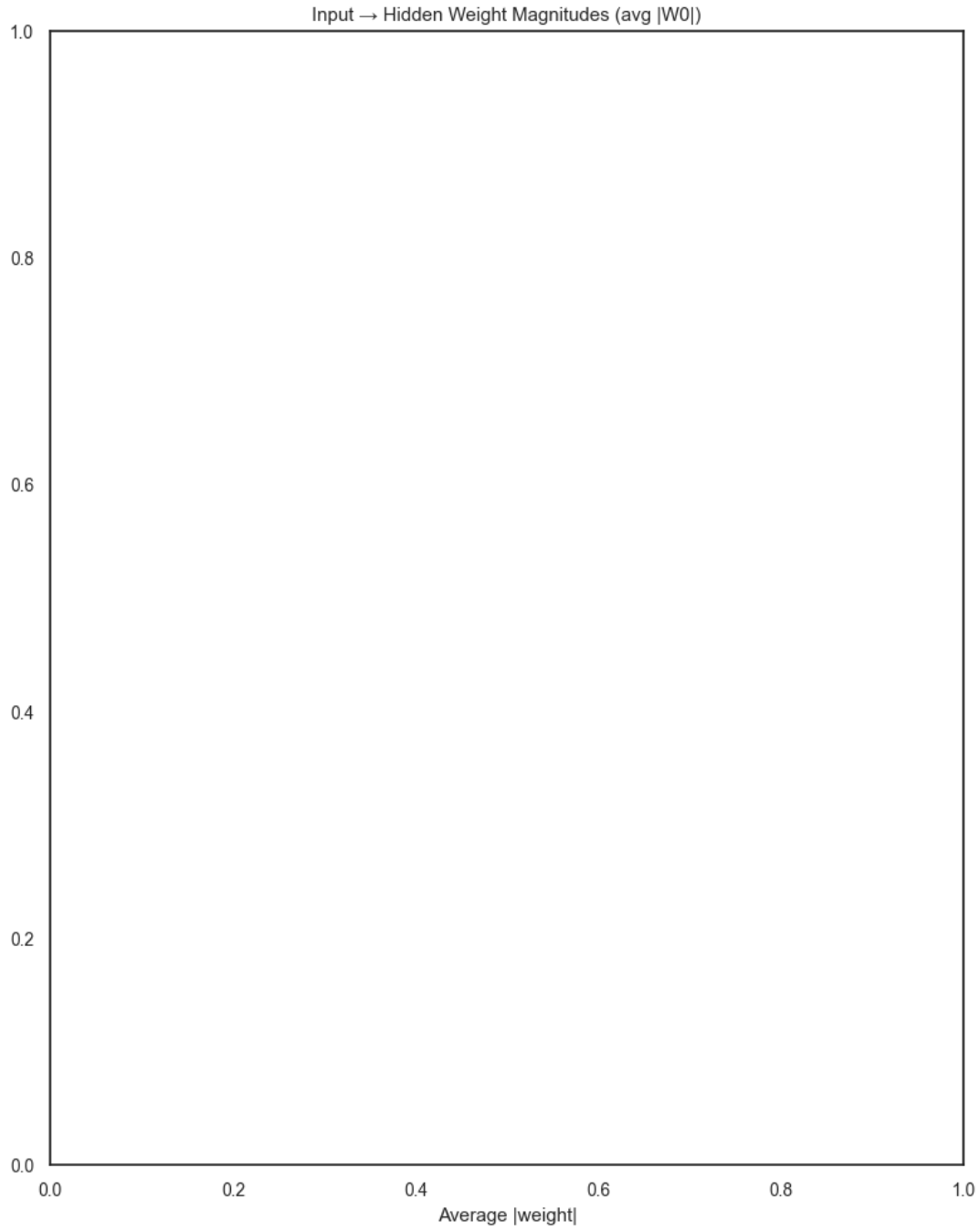
```
Training RMSE: 6.951501409137495
Coefficient of determination (R²) on training data: 0.358
Test RMSE: 4.555392989431061
Coefficient of determination (R²) on test data: 0.5398
Validation RMSE: 8.564928331625294
Coefficient of determination (R²) on  validation data: 0.2628
```



[43]:
```
# #Examine weights from the NNR
# import io, contextlib

# buf = io.StringIO()
# with contextlib.redirect_stdout(buf):  # hide print outputs
#     res = neural_network_regression("WLTP CO2", data_fuel);

# # from your Regressor() output
# W0 = res["W0"]                    # shape: [n_features, H]
# features = res["features"]
# imp = np.mean(np.abs(W0), axis=1)

# print("W0 shape:", W0.shape)
# print("Number of features:", len(features))

# # Ensure they match in length
# min_len = min(len(features), len(imp))
# features = features[:min_len]
# imp = imp[:min_len]

# imp_df = pd.DataFrame({"Feature": features, "Importance": imp})
# imp_df = imp_df.sort_values(by="Importance", ascending=True)
```

[44]:
```
# importance = average absolute incoming weight to hidden layer
plt.figure(figsize=(8, 10))
#plt.barh(imp_df["Feature"], imp_df["Importance"])
plt.title("Input → Hidden Weight Magnitudes (avg |W0|)")
```

```
plt.xlabel("Average |weight|")
plt.tight_layout()
plt.show()
```



3.7.2 Neural Network Regression - 3 Layers

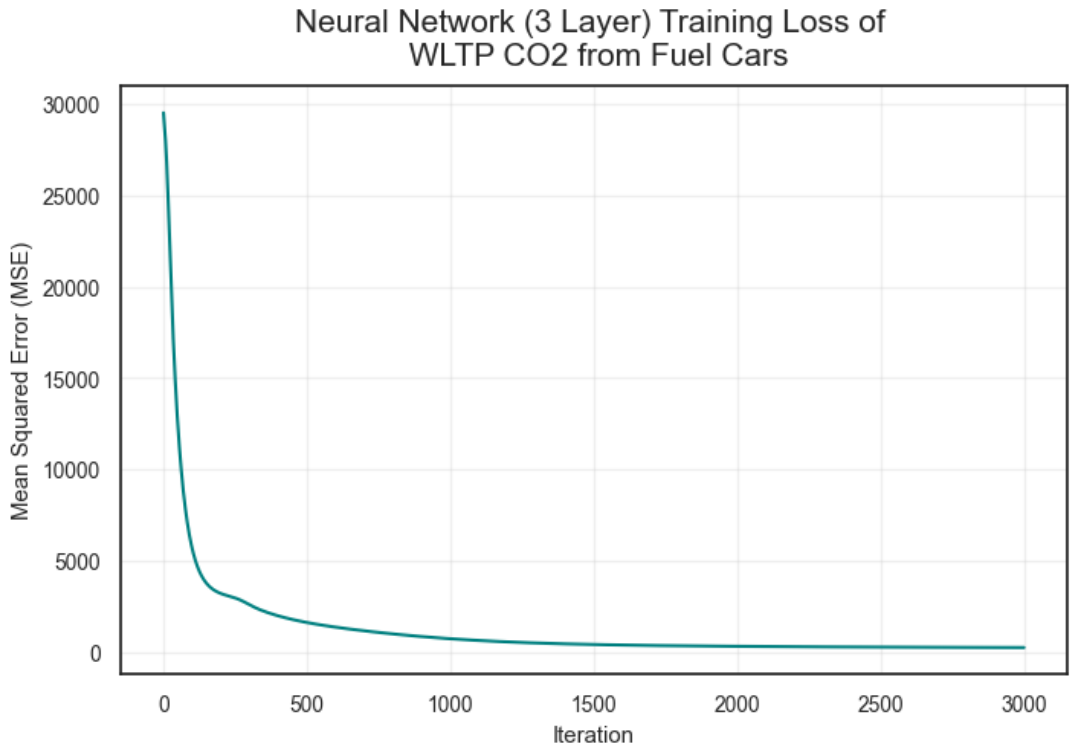
[45]:
```
# Reusable function for Neural Network regression task - 2 hidden layer NN
def neural_network_regression_2hidden(target: str, data: pd.DataFrame,␣
 ↪random_state: int = 42):
    assert target in data.columns, f"Target '{target}' not in dataframe."

    splits = data_split(target, data)

    X_train = splits["X_train"]
    X_val   = splits["X_val"]
    X_test  = splits["X_test"]
    y_train = splits["y_train"]
    y_val   = splits["y_val"]
    y_test  = splits["y_test"]

    # Flatten target arrays
    y_train = y_train.values.reshape(-1, 1)
    y_val   = y_val.values.reshape(-1, 1)
    y_test  = y_test.values.reshape(-1, 1)

    # scaler = StandardScaler()
    # X_train_scaled = scaler.fit_transform(X_train)
    # X_test_scaled = scaler.transform(X_test)
    # X_val_scaled = scaler.transform(X_val)

    # Define NN dimensions
    N = len(y_train) # number of samples
    D_in = X_train.shape[1] # number of features
    D_out = 1 # number of outputs
    H1 = 30 # number of hidden neurons in hidden layer 1
    H2 = 20 # number of hidden neurons in hidden layer 2

    # -- Initialize weights
    # Layer 1: input -> hidden1
    W0 = normal(0, 0.1, size=(D_in, H1))
    b0 = np.zeros((1, H1))

    # Layer 2: hidden1 -> hidden2
    W1 = normal(0, 0.1, size=(H1, H2))
    b1 = np.zeros((1, H2))

    # Output: hidden2 -> output
    W2 = normal(0, 0.1, size=(H2, D_out))
    b2 = np.zeros((1, D_out))

    # Training loop
    eta = 0.0003

    loss_history = []
    for i in range(3001):
```

```
        # ---- forward pass ----
        ## Layer 1
        z1 = X_train.dot(W0) + b0
        z1 = np.clip(z1, -500, 500)
        a1 = 1.0 / (1.0 + np.exp(-z1)) # sigmoid

        # Layer 2
        z2 = a1.dot(W1) + b1
        z2 = np.clip(z2, -500, 500)
        a2 = 1.0 / (1.0 + np.exp(-z2)) # sigmoid

        # Output (linear)
        y_hat = a2.dot(W2) + b2

        # Loss (MSE)
        Loss = np.mean((y_hat - y_train) ** 2)
        loss_history.append(float(Loss))

        # ---- backward pass ----
        # dLoss/dy_hat = 2*(y_hat - y) / N
        dy = (2.0 / N) * (y_hat - y_train)

        # Gradients for output layer (a2 -> y_hat)
        dW2 = a2.T.dot(dy)
        db2 = np.sum(dy, axis=0, keepdims=True)

        # Backprop into hidden2
        da2 = dy.dot(W2.T)
        dz2 = da2 * a2 * (1.0 - a2)
        dW1 = a1.T.dot(dz2)
        db1 = np.sum(dz2, axis=0, keepdims=True)

        # Backprop into hidden1
        da1 = dz2.dot(W1.T)
        dz1 = da1 * a1 * (1.0 - a1)
        dW0 = X_train.T.dot(dz1)
        db0 = np.sum(dz1, axis=0, keepdims=True)

        # ---- update parameters ----
        W2 -= eta * dW2
        b2 -= eta * db2
        W1 -= eta * dW1
        b1 -= eta * db1
        W0 -= eta * dW0
        b0 -= eta * db0

    # # Evaluate
    # z1 = X_train_scaled.dot(W0)
    # z1 = np.clip(z1, -500, 500)
    # a1 = 1 / (1 + np.exp(-z1))
    # y_pred = a1.dot(W2)
```

```
    # Evaluate on training set
    z1_ = X_train.dot(W0) + b0
    z1_ = np.clip(z1_, -500, 500)
    a1_ = 1.0 / (1.0 + np.exp(-z1_))

    z2_ = a1_.dot(W1) + b1
    z2_ = np.clip(z2_, -500, 500)
    a2_ = 1.0 / (1.0 + np.exp(-z2_))

    y_pred = a2_.dot(W2) + b2

    # MSE and RMSE on training data
    print('Training RMSE:', np.sqrt(mean_squared_error(y_train, y_pred)))
    print('Coefficient of determination (R²) on training data:',␣
↪round(r2_score(y_train, y_pred),4))

    # Evaluate on test set
    z1_ = X_test.dot(W0) + b0
    z1_ = np.clip(z1_, -500, 500)
    a1_ = 1.0 / (1.0 + np.exp(-z1_))
    z2_ = a1_.dot(W1) + b1
    z2_ = np.clip(z2_, -500, 500)
    a2_ = 1.0 / (1.0 + np.exp(-z2_))
    y_test_pred = a2_.dot(W2) + b2

    print('Test RMSE:', np.sqrt(mean_squared_error(y_test, y_test_pred)))
    print('Coefficient of determination (R²) on test data:',␣
↪round(r2_score(y_test, y_test_pred),4))

    # Evaluate on validation set
    z1_ = X_val.dot(W0) + b0
    z1_ = np.clip(z1_, -500, 500)
    a1_ = 1.0 / (1.0 + np.exp(-z1_))
    z2_ = a1_.dot(W1) + b1
    z2_ = np.clip(z2_, -500, 500)
    a2_ = 1.0 / (1.0 + np.exp(-z2_))
    y_val_pred = a2_.dot(W2) + b2

    print('Validation RMSE:', np.sqrt(mean_squared_error(y_val, y_val_pred)))
    print('Coefficient of determination (R²) on  validation data:',␣
↪round(r2_score(y_val, y_val_pred),4))


    # Auto labeling based on Fuel Type column
    if data["Fuel Type"].isin(fuel_types).any():
        data_label = "Fuel Cars"
    elif data["Fuel Type"].isin(hybrid_types).any():
        data_label = "Hybrid Cars"
    else:
```

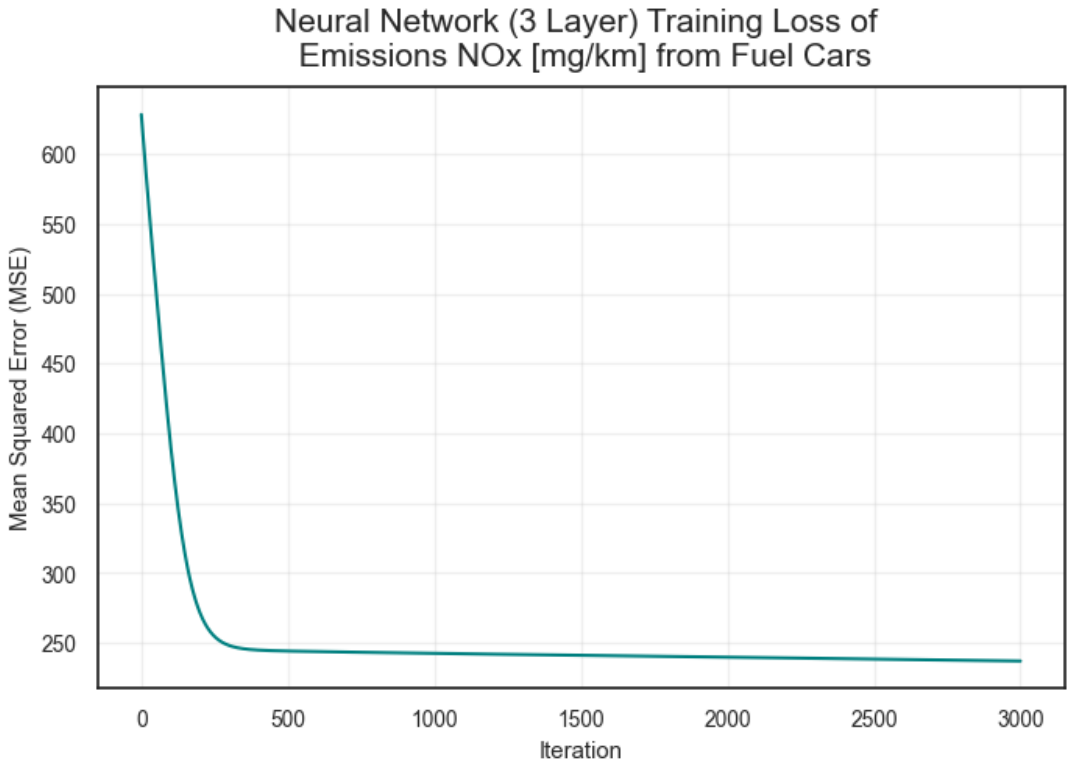
```
        data_label = "Dataset"

    #Visualise the MSE
    plt.figure(figsize=(8,5))
    plt.plot(loss_history, color='teal')
    plt.title(f"Neural Network (3 Layer) Training Loss of \n {target} from␣
 ↪{data_label}",
              fontsize = 15, pad = 10)
    plt.xlabel("Iteration")
    plt.ylabel("Mean Squared Error (MSE)")
    plt.grid(alpha=0.3)
    plt.show()

    # # Return the weights and features of the model
    # return{
    #     "features": features,
    #     "W0": W0
    #     }

# Neural Network regression for fuel cars, target variable: WLTP CO2
neural_network_regression_2hidden("WLTP CO2", data_fuel)
```

```
Training RMSE: 15.622564437897724
Coefficient of determination (R²) on training data: 0.92
Test RMSE: 15.850467231513218
Coefficient of determination (R²) on test data: 0.888
Validation RMSE: 18.690134370355924
Coefficient of determination (R²) on  validation data: 0.904
```



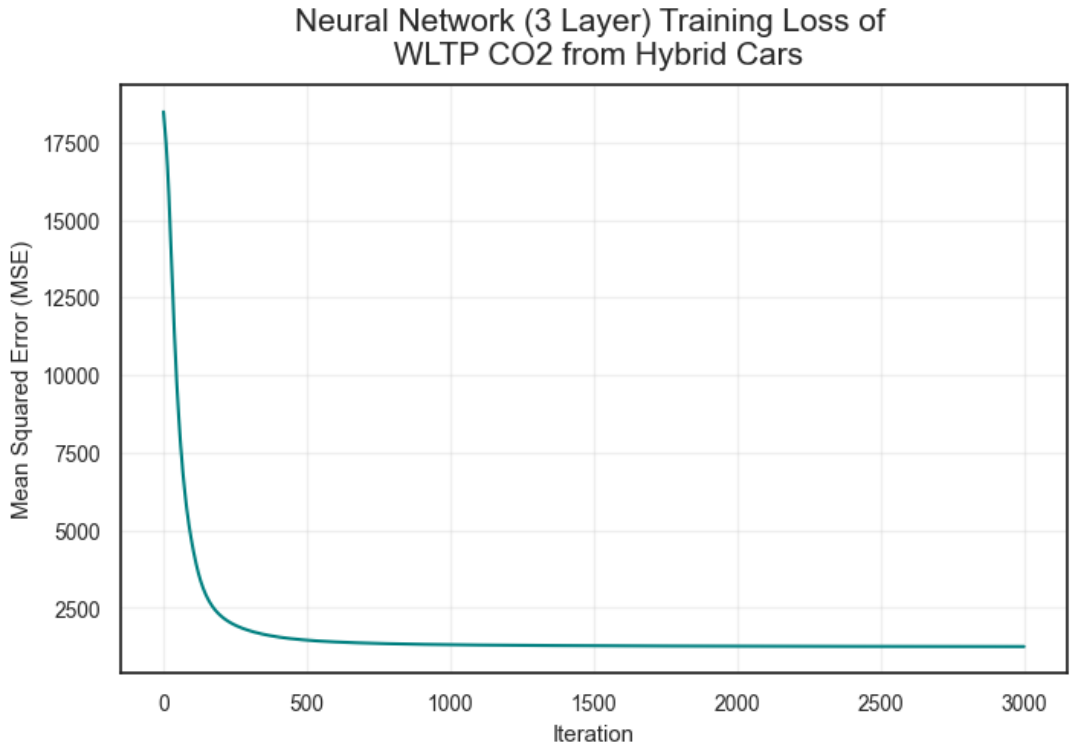
[46]:
```
# NN Regression (2 hidden layers), Emissions NOx from Fuel cars
neural_network_regression_2hidden("Emissions NOx [mg/km]", data_fuel)
```

```
Training RMSE: 15.393607895371947
Coefficient of determination (R²) on training data: 0.0338
Test RMSE: 12.28769373326525
Coefficient of determination (R²) on test data: -0.0088
Validation RMSE: 12.82621176054791
Coefficient of determination (R²) on  validation data: 0.0358
```



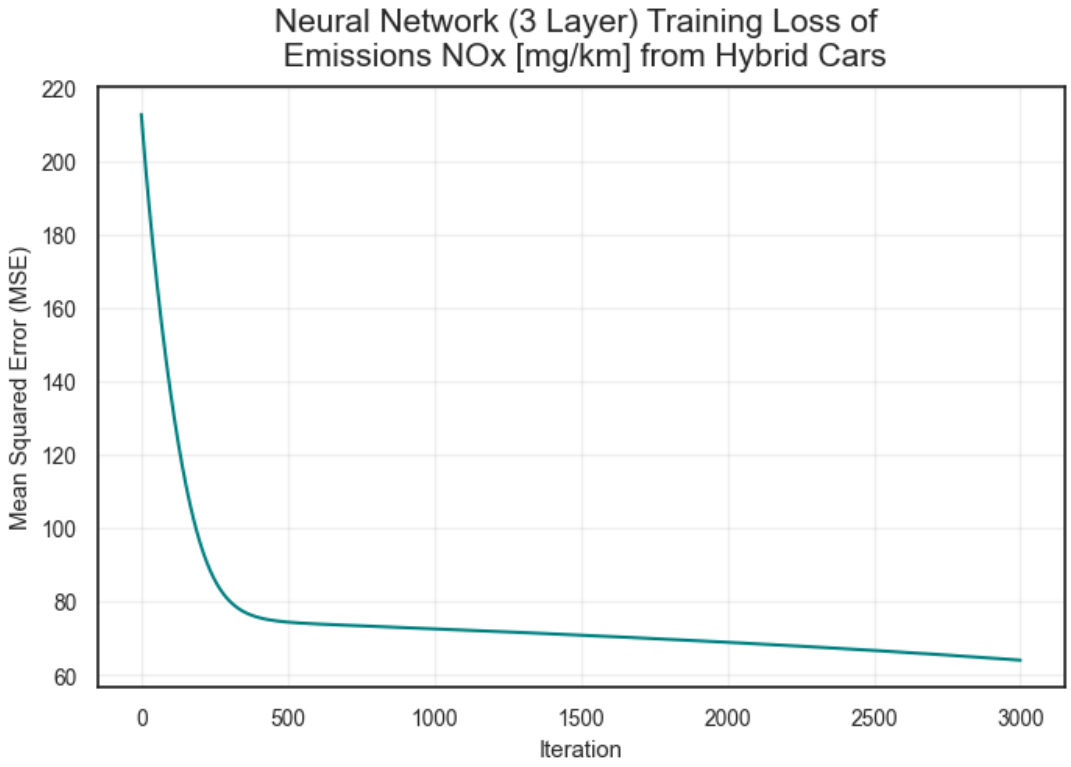
[47]:
```
# NN Regression (2 hidden layers), CO2 emissions from Hybrid cars
neural_network_regression_2hidden("WLTP CO2", data_hybrid)
```

```
Training RMSE: 35.42839109030671
Coefficient of determination (R²) on training data: 0.5719
Test RMSE: 32.62246511055373
Coefficient of determination (R²) on test data: 0.6169
Validation RMSE: 32.63765510527334
Coefficient of determination (R²) on  validation data: 0.4737
```



[48]:
```
# NN Regression (2 hidden layers), Emissions NOx from Hybrid cars
neural_network_regression_2hidden("Emissions NOx [mg/km]", data_hybrid)
```

```
Training RMSE: 8.005424250010037
Coefficient of determination (R²) on training data: 0.1486
Test RMSE: 5.914581063728446
Coefficient of determination (R²) on test data: 0.2243
Validation RMSE: 9.415363072073598
Coefficient of determination (R²) on  validation data: 0.1092
```



3.7.3 Neural Network Regression - 4 Layers

[49]:
```
# Reusable function for Neural Network regression task - 3 hidden layer NN

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def neural_network_regression_3hidden(target: str, data: pd.DataFrame,␣
 ↪random_state: int = 42):
    assert target in data.columns, f"Target '{target}' not in dataframe."

    # Perform data splitting
    splits = data_split(target, data)

    X_train = splits["X_train"]
    X_val   = splits["X_val"]
    X_test  = splits["X_test"]
    y_train = splits["y_train"]
    y_val   = splits["y_val"]
    y_test  = splits["y_test"]

    # Flatten target arrays
    y_train = y_train.values.reshape(-1, 1)
    y_val   = y_val.values.reshape(-1, 1)
    y_test  = y_test.values.reshape(-1, 1)
```

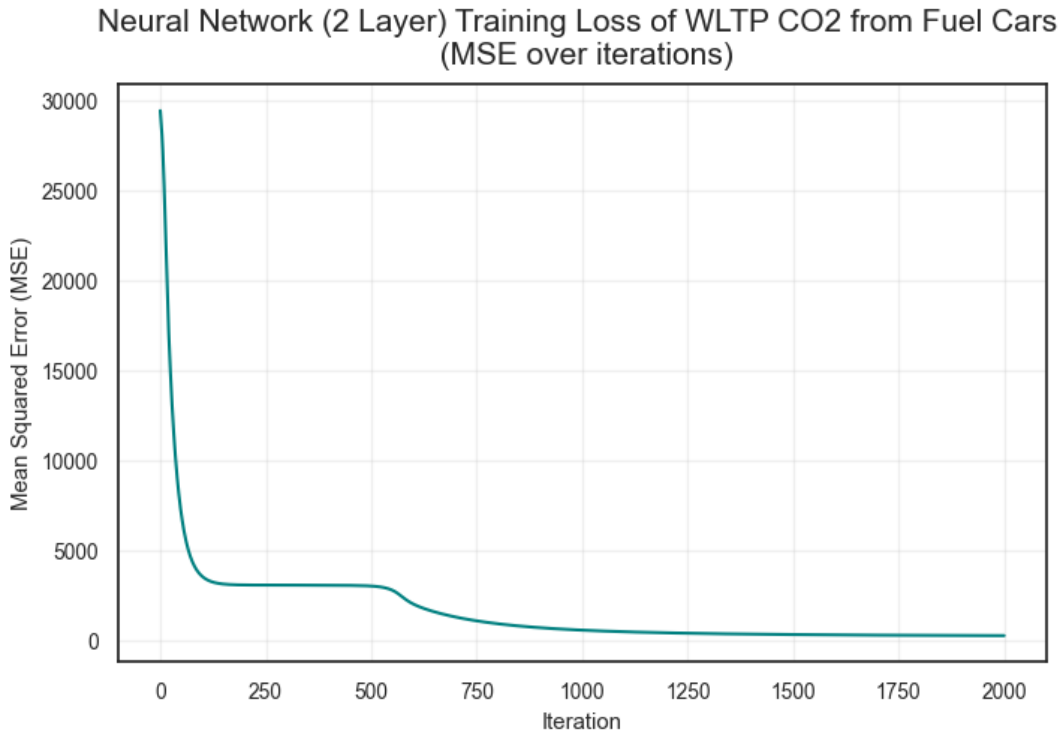
```
# Define NN dimensions
N = len(y_train) # number of samples
D_in = X_train.shape[1] # number of features
D_out = 1 # number of outputs

H1 = 40 # number of hidden neurons in hidden layer 1
H2 = 30 # number of hidden neurons in hidden layer 2
H3 = 20 # number of hidden neurons in hidden layer 3

# -- Initialize weights
# Layer 1: input -> hidden1
W0 = normal(0, 0.1, size=(D_in, H1))
b0 = np.zeros((1, H1))

# Layer 2: hidden1 -> hidden2
W1 = normal(0, 0.1, size=(H1, H2))
b1 = np.zeros((1, H2))

# Layer 3: hidden2 -> hidden3
W2 = normal(0, 0.1, size=(H2, H3))
b2 = np.zeros((1, H3))

# Output: hidden3 -> output
W3 = normal(0, 0.1, size=(H3, D_out))
b3 = np.zeros((1, D_out))

# Training loop
eta = 0.0005

loss_history = []
for i in range(2001):
    # ---- forward pass ----
    ## Layer 1
    z1 = X_train.dot(W0) + b0
    z1 = np.clip(z1, -500, 500)
    a1 = 1 / (1 + np.exp(-z1)) # sigmoid

    # Layer 2
    z2 = a1.dot(W1) + b1
    z2 = np.clip(z2, -500, 500)
    a2 = 1 / (1 + np.exp(-z2)) # sigmoid

    # Layer 3
    z3 = a2.dot(W2) + b2
    z3 = np.clip(z3, -500, 500)
    a3 = 1 / (1 + np.exp(-z3)) # sigmoid

    # Output (linear)
    y_hat = a3.dot(W3) + b3
```

```
        # Loss (MSE)
        Loss = np.mean((y_hat - y_train) ** 2)
        loss_history.append(float(Loss))


        # ---- backward pass ----
        dy = (2.0 / N) * (y_hat - y_train)

        # Gradients for output layer (a3 -> y_hat)
        dW3 = a3.T.dot(dy)
        db3 = np.sum(dy, axis=0, keepdims=True)

        # Backprop into hidden3
        da3 = dy.dot(W3.T)
        dz3 = da3 * a3 * (1 - a3)
        dW2 = a2.T.dot(dz3)
        db2 = np.sum(dz3, axis=0, keepdims=True)

        # Backprop into hidden2
        da2 = dz3.dot(W2.T)
        dz2 = da2 * a2 * (1 - a2)
        dW1 = a1.T.dot(dz2)
        db1 = np.sum(dz2, axis=0, keepdims=True)

        # Backprop into hidden1
        da1 = dz2.dot(W1.T)
        dz1 = da1 * a1 * (1 - a1)
        dW0 = X_train.T.dot(dz1)
        db0 = np.sum(dz1, axis=0, keepdims=True)

        # ---- update parameters ----
        W3 -= eta * dW3
        b3 -= eta * db3

        W2 -= eta * dW2
        b2 -= eta * db2

        W1 -= eta * dW1
        b1 -= eta * db1

        W0 -= eta * dW0
        b0 -= eta * db0

    # Evaluate on training set
    def predict(X):
        z1 = X.dot(W0) + b0
        z1_ = np.clip(z1, -500, 500)
        a1 = 1.0 / (1.0 + np.exp(-z1_))

        z2 = a1.dot(W1) + b1
```

```
        z2_ = np.clip(z2, -500, 500)
        a2 = np.clip(z2_, -500, 500)

        z3 = a2.dot(W2) + b2
        z3_ = np.clip(z3, -500, 500)
        a3 = 1.0 / (1.0 + np.exp(-z3_))

        return a3.dot(W3) + b3


    # z1 = X_train_scaled.dot(W0) + b0
    # z1_ = np.clip(z1, -500, 500)
    # a1 = 1.0 / (1.0 + np.exp(-z1_))

    # z2 = a1.dot(W1) + b1
    # z2_ = np.clip(z2, -500, 500)
    # a2 = np.clip(z2_, -500, 500)

    # z3 = a2.dot(W2) + b2
    # z3_ = np.clip(z3, -500, 500)
    # a3 = 1.0 / (1.0 + np.exp(-z3_))
    # y_train_pred = a3.dot(W3) + b3

    # MSE and RMSE on training data
    y_train_pred = predict(X_train)
    print('Training RMSE:', np.sqrt(mean_squared_error(y_train,␣
↪y_train_pred)))
    print('Coefficient of determination (R²) on training data:',␣
↪round(r2_score(y_train, y_train_pred),4))

    # Evaluate on test set
    y_test_pred = predict(X_test)
    print('Test RMSE:', np.sqrt(mean_squared_error(y_test, y_test_pred)))
    print('Coefficient of determination (R²) on test data:',␣
↪round(r2_score(y_test, y_test_pred),4))

    # Evaluate on validation set
    y_val_pred = predict(X_val)
    print('Validation RMSE:', np.sqrt(mean_squared_error(y_val, y_val_pred)))
    print('Coefficient of determination (R²) on  validation data:',␣
↪round(r2_score(y_val, y_val_pred),4))


    # Auto labeling based on Fuel Type column
    if data["Fuel Type"].isin(fuel_types).any():
        data_label = "Fuel Cars"
    elif data["Fuel Type"].isin(hybrid_types).any():
        data_label = "Hybrid Cars"
    else:
        data_label = "Dataset"
```

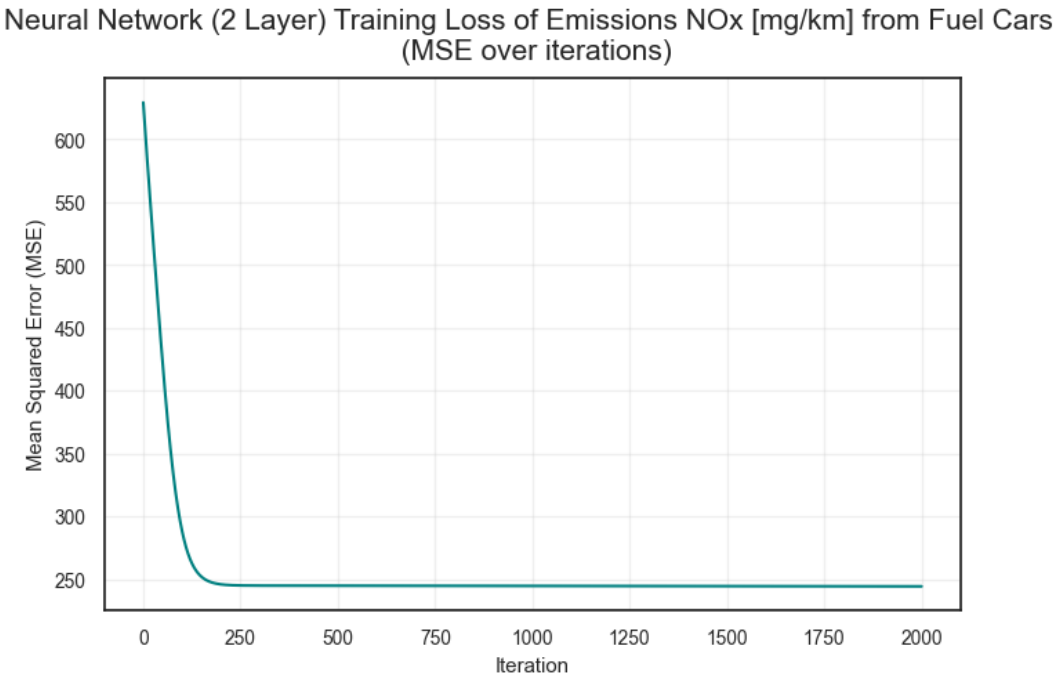
```
    #Visualise the MSE
    plt.figure(figsize=(8,5))
    plt.plot(loss_history, color='teal')
    plt.title(f"Neural Network (2 Layer) Training Loss of {target} from␣
 ↪{data_label} \n (MSE over iterations)",
              fontsize = 15, pad = 10)
    plt.xlabel("Iteration")
    plt.ylabel("Mean Squared Error (MSE)")
    plt.grid(alpha=0.3)
    plt.show()

    # # Return the weights and features of the model
    # return{
    #     "features": features,
    #     "W0": W0
    #     }

# Neural Network regression for fuel cars, target variable: WLTP CO2
neural_network_regression_3hidden("WLTP CO2", data_fuel)
```

```
Training RMSE: 138.1645823541116
Coefficient of determination (R²) on training data: -5.2572
Test RMSE: 138.67875772052057
Coefficient of determination (R²) on test data: -7.5738
Validation RMSE: 146.49132268610106
Coefficient of determination (R²) on  validation data: -4.8955
```



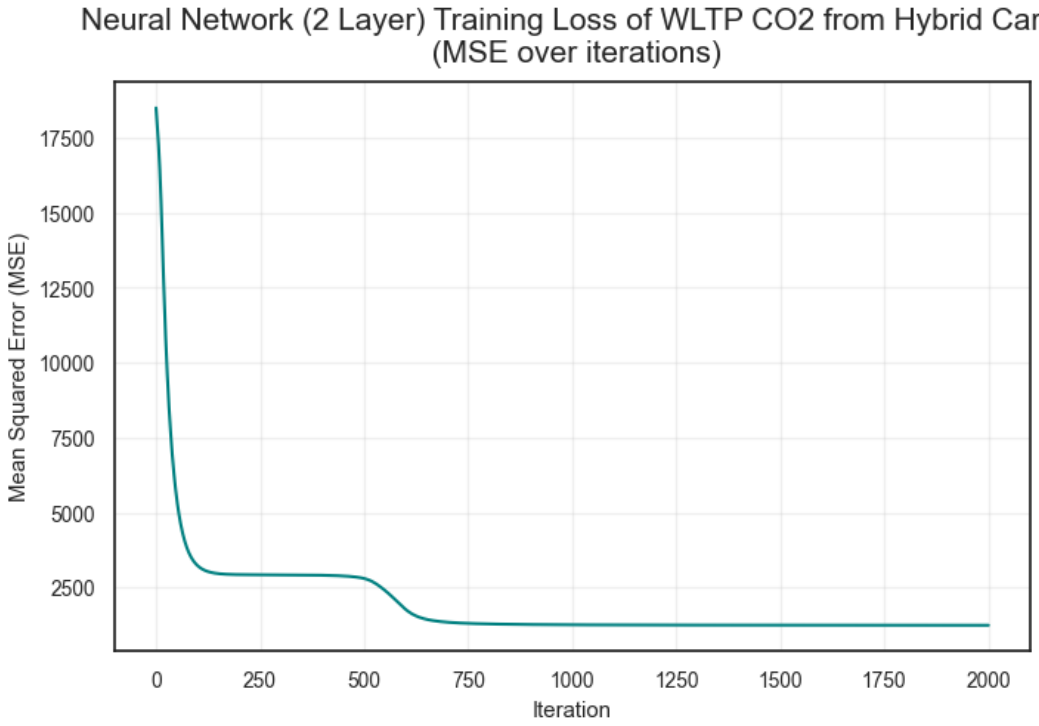
[50]:
```
neural_network_regression_3hidden("Emissions NOx [mg/km]", data_fuel)
```

```
Training RMSE: 15.601972032883996
Coefficient of determination (R²) on training data: 0.0075
Test RMSE: 12.140231683389397
Coefficient of determination (R²) on test data: 0.0153
Validation RMSE: 12.977859002448692
Coefficient of determination (R²) on  validation data: 0.0129
```



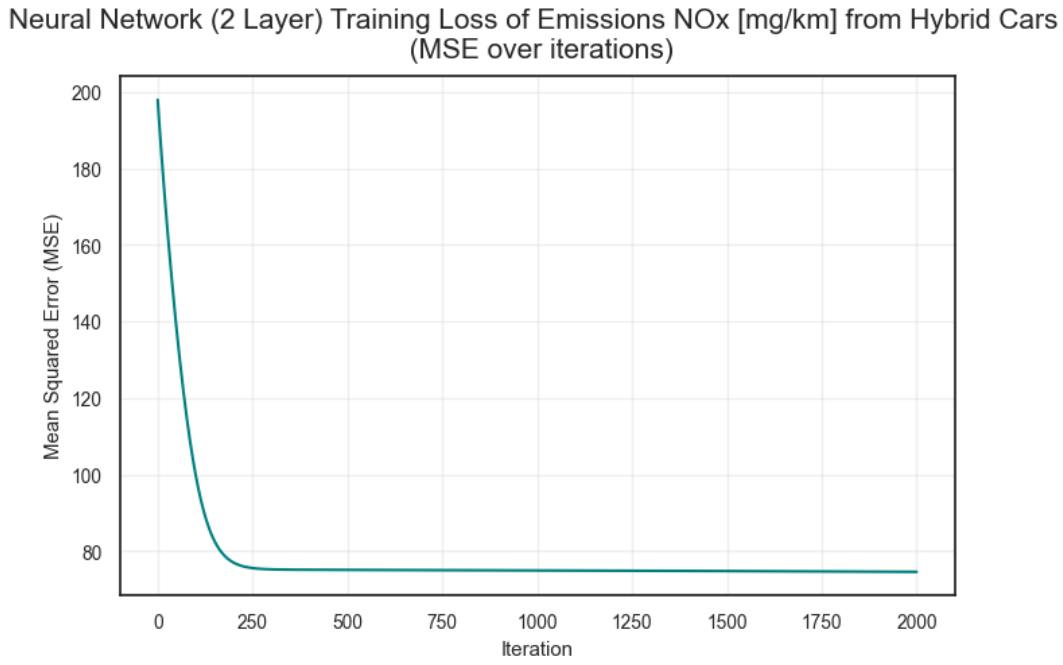
[51]:
```
neural_network_regression_3hidden("WLTP CO2", data_hybrid)
```

```
Training RMSE: 35.56435477602406
Coefficient of determination (R²) on training data: 0.5687
Test RMSE: 32.2759250197809
Coefficient of determination (R²) on test data: 0.625
Validation RMSE: 32.67101399065757
Coefficient of determination (R²) on  validation data: 0.4726
```



[52]:
```
neural_network_regression_3hidden("Emissions NOx [mg/km]", data_hybrid)
```

```
Training RMSE: 8.549650805680743
Coefficient of determination (R²) on training data: 0.0289
Test RMSE: 6.486216803484162
Coefficient of determination (R²) on test data: 0.0671
Validation RMSE: 9.968267522196147
Coefficient of determination (R²) on  validation data: 0.0015
```



4 References

- European Environment Agency, 2023, 'Greenhouse Gas Emissions from Transport in Europe.', *EEA Report No. 23/2023*, Publications Office of the European Union.
- Intergovernmental Panel on Climate Change (IPCC), 2022, 'Climate Change 2022: Mitigation of Climate Change', *Contribution of Working Group III to the Sixth Assessment Report of the IPCC*, Cambridge University Press.
- UK Vehicle Certification Agency, 2024, *Fuel Consumption and Emission Data Tables: Car Fuel and CO Information. Department for Transport*, viewed 29 September 2025, https://www.vehicle-certification-agency.gov.uk/
- World Health Organization, 2021, *WHO Global Air Quality Guidelines: Particulate Matter, Ozone, Nitrogen Dioxide, Sulfur Dioxide and Carbon Monoxide*, Geneva: WHO.
- U.S. Environmental Protection Agency (EPA), 2023, *Nitrogen Oxides (NO) Pollution*, EPA.
- European Environment Agency (EEA), 2022, *Air Pollution Sources — Volatile Organic Compounds (VOCs)*, EEA Report
- Veza, Ibham et al, 2023, 'Electric Vehicle (EV) and Driving Towards Sustainability: Comparison between EV, HEV, PHEV, and ICE Vehicles to Achieve Net Zero Emissions by 2050 from EV', *Alexandria Engineering Journal*, vol. 82, pp. 459–467, DOI.10.1016/j.aej.2023.10.020